

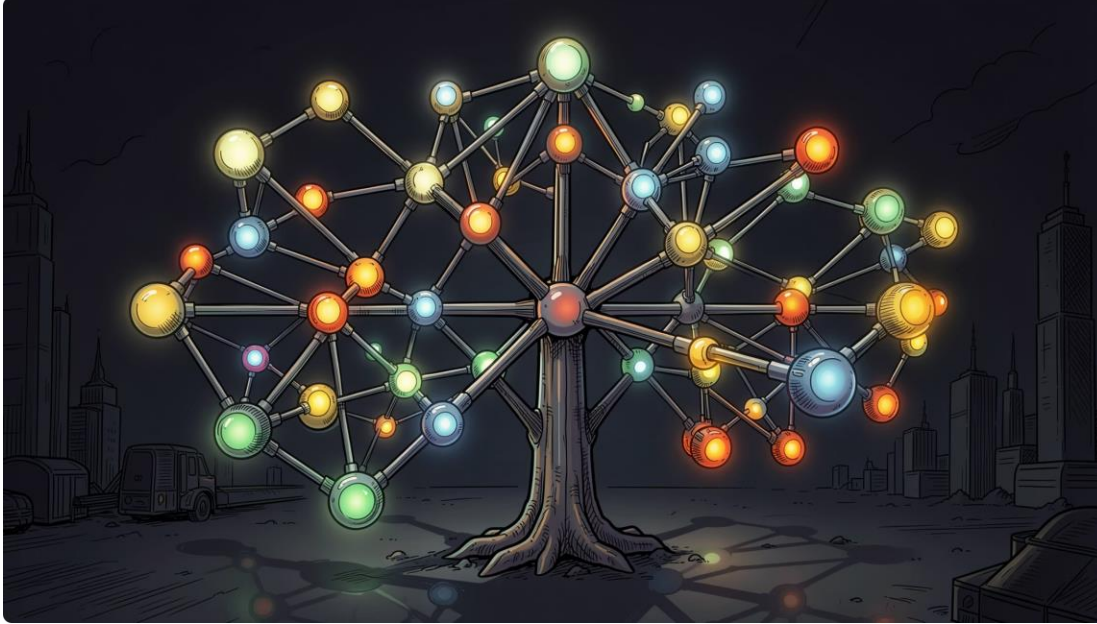
B ve B+ Ağaçlarının Modern Sistemlerdeki Rolü

Veri Tabanlarından Büyük Veriye, Bilgiyi Organize Etme Yolu

AHMET KAN KAHRAMAN · 02240224161



Teorik Temeller: B ve B+ Ağaçları Nedir?



B-Tree

Kendi kendini dengeleyen çok yollu arama ağacıdır. Her düğüm (node) hem **indeks anahtarı** hem de **gerçek veriyi** barındırır. Veri herhangi bir katmanda bulunabilir.

B+ Tree

İç düğümler yalnızca **yönlendirici indeks anahtarları** tutar. Tüm gerçek veriler en alt katmandaki yapraklarda (leaf nodes) yer alır ve bu yapraklar birbirine **bağlı liste (Linked List)** ile zincirlenmiştir.

❗ Her iki yapı da $O(\log n)$ karmaşıklığıyla çalışır; ancak B+ Tree aralık sorgularında belirgin üstünlük sağlar.



Günlük Hayattan Örnek: Akıllı Hastane Arşivi

B-Tree Arşivi 🗄️

Her dosya dolabında hem **yönlendirme etiketleri** hem de **hasta dosyaları** bulunur. Aradığınız bilgiye herhangi bir çekmecede ulaşabilirsiniz — ama tüm kat boyunca tek tek bakmanız gerekebilir.

B+ Tree Arşivi 🏢

Üst katlarda yalnızca **yönlendirme kartları** bulunur. Tüm hasta dosyaları bodrum katta, alfabetik sırayla ve birbirine **zincirli** biçimde dizilmiştir. Bir kez kata inince sıralı erişim çok hızlıdır.

Bu Örneğin Faydası: Aralık Sorguları



Nokta Sorgusu

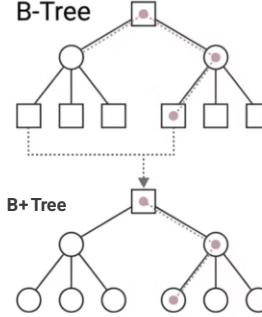
Tek bir hastayı ararken her iki sistem de **3-4 adımda** hedefe ulaşır.
Performans farkı minimumdur.



Aralık Sorgusu (Range Query)

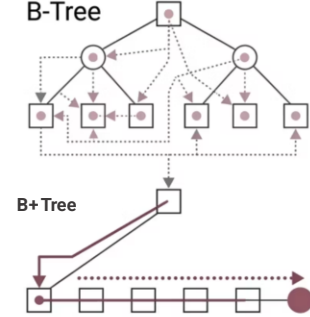
"**K ile M arasındaki tüm dosyaları getir**" komutuyla B+ Tree bodrum kata iner ve zincirli yaprak listesi boyunca **tek seferde düz bir çizgide** tüm kayıtları toplar.

Nokta Sorgu



Her ikisi de 3-4
adımda hedefe
ulaşır, eşit
performans.

Aralık Sorgu



B-Tree çoklu yol izler;
B+ Tree tek iniş ve
bağlı yaprakileçok daha
hızlı.

Veri Tabanı Motorlarındaki Kullanımı



MySQL

Her bir tablosu başlıca bir **B+ Tree**'dir. Birincil anahtar, tablonun fiziksel sırasını belirler.



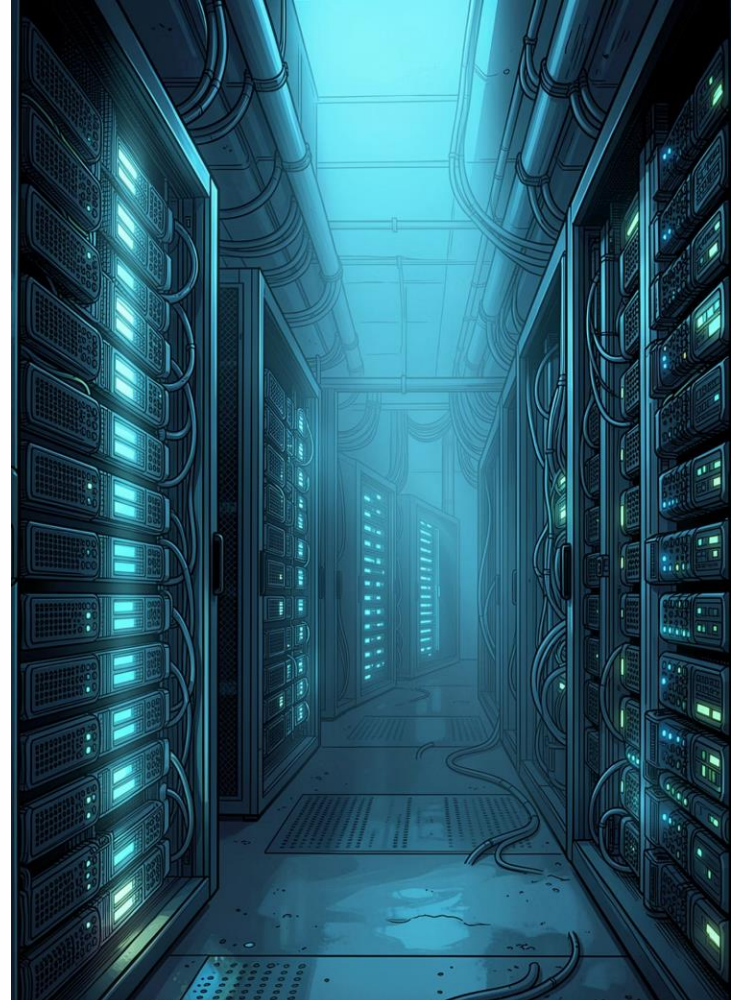
PostgreSQL

MVCC mimarisiyle yüksek eşzamanlılığa izin veren, kilitsiz okuma için optimize edilmiş gelişmiş B-Tree implementasyonu.



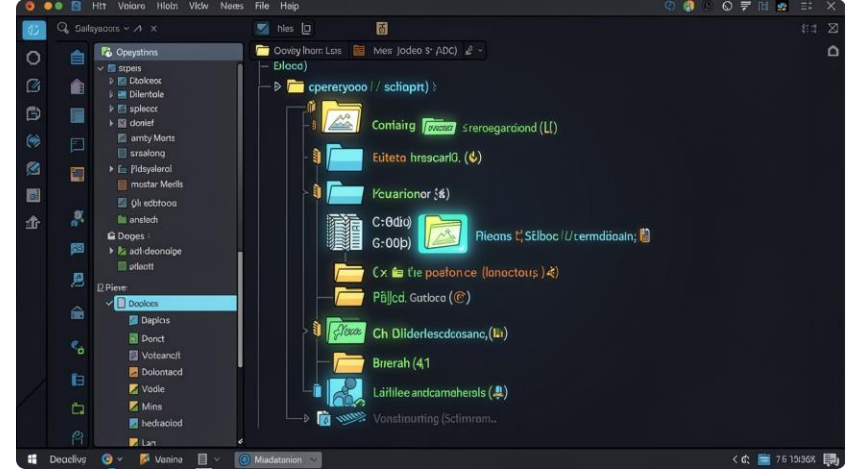
MongoDB

NoSQL belge modelini benimsemesine karşın, tüm koleksiyon indekslerini B-Tree üzerine inşa eder.



Dosya Sistemlerindeki Yeri

İşletim sistemleri, klasör hiyerarşilerini ve metadata'yı yönetmek için onlarca yıldır ağaç yapılarına güvenmektedir. Milyonlarca dosyayı barındıran disklerde dizin arama süresi sabit kalır.



NTFS

Windows'un varsayılan dosya sistemi;
MFT kaydını B-Tree ile indeksler.

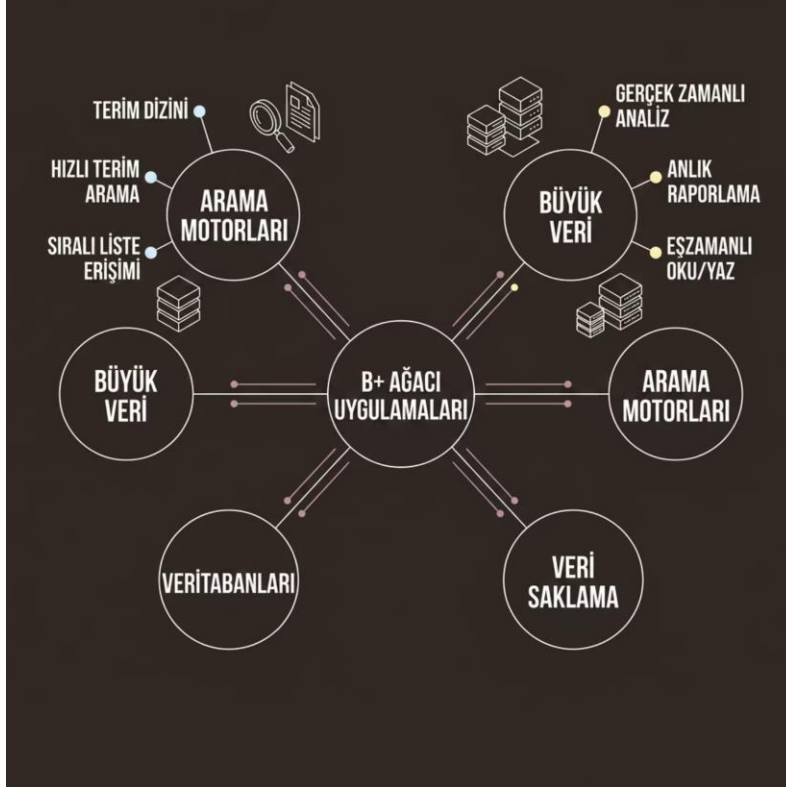
EXT4 · H-Tree

Linux çekirdeğinin dizin aramalarını
hızlandıran hash tabanlı B-Tree türevidir.

APFS

Apple'ın modern dosya sistemi; anlık
görüntü ve şifrelemeyi B-Tree kopyalama
semantiğiyle destekler.

Arama Motorları ve Büyük Veri



Arama Motorları

Ters indeksteki (Inverted Index) **terim sözlükleri**, B+ Tree üzerinde tutulur. "machine learning" araması yapıldığında sistem ağacın içinden terimi milisaniyeler içinde bulur.

Büyük Veri (Big Data)

Gerçek zamanlı analiz (Real-time Analytics) sistemleri, B+ Tree'nin **kilitsiz eşzamanlı okuma** özelliğinden yararlanarak sistemi durdurmadan anlık raporlar üretebilir.

 Google arama motoru bu yapı (B-tree) ile çalışır.

Modern Sistemlerde Neden Tercih Edilir?



Disk I/O Optimizasyonu

Disk blok boyutuna tam oturan düğümler sayesinde tek bir okumada **yüzlerce indeks** RAM'e çekilir.



$O(\log n)$ Garantisi

Milyarlarca kayıta bile ağaç derinliği **sığ kalır**; erişim süresi öngörülebilir ve kararlıdır.

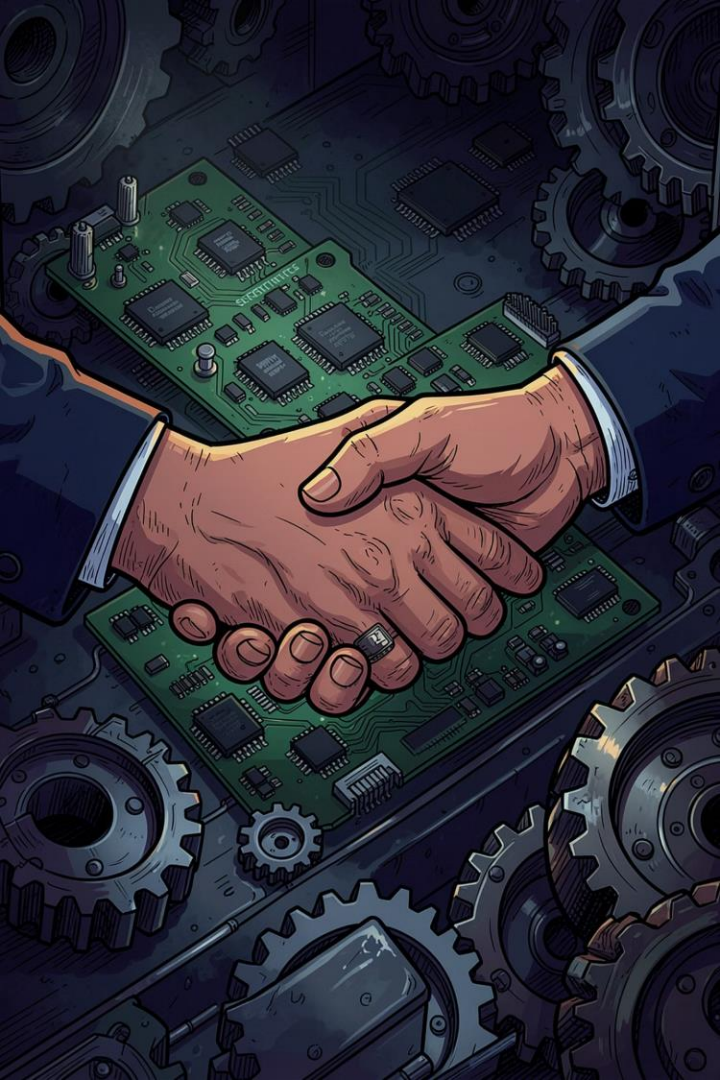


Önbellek Verimliliği

Kök ve üst düğümler **RAM'de önbelleğe** alınır; en sık kullanılan yollar neredeyse sıfır gecikmeyle yanıt verir.



Bu üç özelliğin birleşimi, B+ Tree'yi hem OLTP hem OLAP iş yüklerinde birden fazla kez "kazanan" mimar yapar.



Neden Bu Kadar Önemli? ve Neden Büyük Veri Çağında İhtiyaç Duyuluyor?

B+ Tree, yazılım ile donanım arasındaki **barış antlaşmasıdır**.

İşlemci hızları nanosaniye ölçeğinde çalışırken mekanik diskler milisaniye gecikmesiyle yanıt verir. Bu uçurum, bilgisayar mimarisinin en kritik darboğazıdır. B+ Tree, düğümlerini disk bloklarına hizalayarak bu gecikmeyi (latency) minimize eden **en olgun mimari çözüm** olarak öne çıkar.

Donanım Gerçeği

Disk erişimi CPU'dan **100.000 kat** yavaştır

B+ Tree Çözümü

Tek I/O'da **yüzlerce** anahtar yüklenir

Sonuç

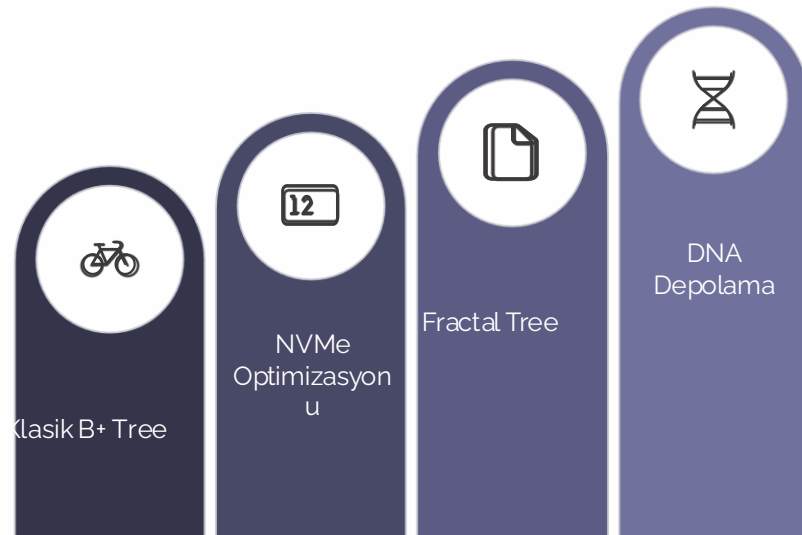
Sistem **öngörülebilir ve kararlı** performans gösterir

Gelecekte de Kullanılacak mı?

Cevap ; Kesinlikle Evet.

LSM-Tree gibi yeni yapılar yazma performansında öne çıksa da, **okuma hızında B+ Tree** hâlâ tartışmasız liderdir. Milyonlarca üretim sisteminde kanıtlanmış güvenilirlik sunar. Gelecekteki DNA/biyolojik veri depolama birimlerinde bile veriyi **adresleme ve indeksleme** zorunlu olacak. **Fractal Tree** gibi paralelleştirilmiş varyasyonlar bu ihtiyaca yanıt verecek.

 NVMe SSD'ler ve paralel mimariler B+ ağaçlarının avantajını daha da artırıyor.



| Mantık değişmeyecek — yalnızca donanıma evrilecek.